

Reviewer Pack — Minimal Tropical Hermitian Rank vs Monotone KW Protocol Cost

Entry 04b7bd164579 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 04:20:09 UTC

Minimal Tropical Hermitian Rank vs Monotone KW Protocol Cost

Entry ID: 04b7bd164579 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 04:20:09 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Communication Complexity

Statement:

[‘For any monotone Boolean function f with n variables, the minimal tropical Hermitian rank of its associated matrix is equal to the communication complexity cost of a Karchmer-Wigderson protocol for f .’, ‘This is equivalent to saying that the cost of an optimal Karchmer-Wigderson protocol for f is $\Theta(\text{minimal tropical Hermitian rank}(f))$.’]

Rationale (proposer’s reasoning):

[‘Tropical geometry provides a framework for studying properties of Boolean functions in a more geometric setting. The tropical Hermitian rank measures the complexity of a function in this setting, which may reveal underlying structures that are not apparent when considering Boolean functions directly. If there is a strong correspondence between the tropical Hermitian rank and communication complexity, it suggests a deep connection between these seemingly distinct areas of mathematics.’]

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8b6eba2a687eb4f4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For the Pearson correlation coefficient between the minimal tropical Hermitian rank and the Karchmer-Wigderson protocol cost, if it is greater than or equal to 0.7, the conjecture will be supported. If any seed yields a correlation coefficient less than 0.5, the conjecture will be falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 5 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical geometry AND communication complexity AND Karchmer-Wigderson protocol - Hermitian rank in tropical geometry AND monotone KW protocol cost - Karchmer-Wigderson protocol cost 0 minimal tropical Hermitian rank

Top relevant hits considered: - [<http://arxiv.org/abs/1207.2443v2>] Tropical Teichmüller and Siegel spaces - [<http://arxiv.org/abs/1204.3875v2>] Tropicalizing vs Compactifying the Torelli morphism - [<http://arxiv.org/abs/1204.6154v2>] Local Tropicalization - [<http://arxiv.org/abs/1610.00298v4>] Khovanskii bases, higher rank valuations and tropical geometry - [<http://arxiv.org/abs/2107.05128v2>] Karchmer-Wigderson Games for Hazard-free Computation

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math

def generate_random_monotone_function(n):
    return [random.choice([0, 1]) for _ in range(2**n - 1)]

def matrix_from_function(f, n):
    A = [[0] * (2**n) for _ in range(2**n)]
    for i in range(2**n):
        for j in range(2**n):
            if f[i] == f[j]:
                A[i][j] = 1
    return A

def tropical_hermitian_rank(A):
    n = len(A)
    rank = 0
    while True:
        found_nonzero = False
        for i in range(n):
            if any(A[i][j] != 0 for j in range(n)):
                pivot_row = i
                found_nonzero = True
                break
```

```

    if not found_nonzero:
        return rank
    rank += 1
    for j in range(n):
        if A[pivot_row][j] != 0:
            for k in range(n):
                A[k][j] = max(A[k][j], A[k][pivot_row] + A[pivot_row][j])

def communication_complexity(f, n):
    def simulate_protocol():
        x = random.randint(0, 2**n - 1)
        y = random.randint(0, 2**n - 1)
        return f[x] == f[y]

    success_count = 0
    for _ in range(100): # Run multiple trials to estimate the probability
        if simulate_protocol():
            success_count += 1
    return success_count / 100

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        f = generate_random_monotone_function(n)
        A = matrix_from_function(f, n)
        rank = tropical_hermitian_rank(A)
        cost = communication_complexity(f, n)

        if rank == 0 or cost == 0:
            continue

        results.append((rank, cost))

    if not results:
        return {
            "metric_name": "Pearson correlation coefficient",
            "metric_value": None,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "No valid instances found"
        }

    ranks = [r for r, _ in results]
    costs = [c for _, c in results]

    mean_rank = sum(ranks) / len(ranks)
    mean_cost = sum(costs) / len(costs)

    correlation_coefficient = (sum((r - mean_rank) * (c - mean_cost) for r, c in results) /
                             math.sqrt(sum((r - mean_rank)**2 for r in ranks) *
                                         sum((c - mean_cost)**2 for c in costs)))

```

```

    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "conjecture_holds": correlation_coefficient >= 0.7,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(r["metric_value"] < 0.5 for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"low correlation\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f2e529a6.py", line 110, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f2e529a6.py", line 67, in run_trial
    A = matrix_from_function(f, n)
        ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f2e529a6.py", line 25, in matrix_from_function
    if f[i] == f[j]:
        ~^^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means that the Pearson correlation coefficient could not be calculated. Therefore, the conjecture cannot be supported or falsified based on the provided information. | next: Re-run the test to ensure it completes successfully and produces the necessary data for calculating the Pearson correlation coefficient.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13954
2	preregistration	ollama_remote	glm4:latest	0	0	9252
3	novelty	ollama_remote	glm4:latest	0	0	8311
4	novelty	ollama_remote	glm4:latest	0	0	9240
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12929
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8435
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11710
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12078
9	judge	ollama_remote	glm4:latest	0	0	13882

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 99791 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/04b7bd164579.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/04b7bd164579.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/04b7bd164579.tar.gz (if generated)